Informe técnico

# Biblioteca Digital

API Gateway y microservicio con FastAPI, PostgreSQL y Docker

---

**Estudiante:** Brayan Chambi

**Tecnología asignada:** FastAPI (Python)

**Fecha:** 18 de octubre de 2026

# 1. Descripción del dominio

La biblioteca de un colegio o de un barrio suele llevar su catálogo en cuadernos o en una hoja de cálculo compartida. Con el tiempo pasa lo de siempre: el mismo libro queda registrado dos veces con datos distintos, nadie sabe con certeza cuántas copias hay disponibles y la lista de socios está en otro archivo, a veces en otra computadora.

Este proyecto propone un sistema pequeño, la **Biblioteca Digital**, que reúne esa información en un solo lugar y la ofrece mediante una API REST. El bibliotecario, o la aplicación que use, puede registrar, consultar, modificar y eliminar libros y socios sin tocar la base de datos directamente.

El alcance se limita a dos entidades. Un **libro** tiene título, autor, ISBN, año y número de copias disponibles; un **socio** tiene nombre completo y correo. El ISBN y el correo son únicos, así que el sistema rechaza los duplicados. El registro de préstamos no entra en esta entrega, pero la estructura permite añadirlo después como una entidad nueva sin modificar las existentes.

### ¿Por qué un gateway delante del servicio?

Porque deja al microservicio fuera del alcance directo de los clientes. Toda petición pasa primero por un único punto donde se controla quién entra (JWT), cuántas veces puede hacerlo (rate limiting) y qué queda registrado en los logs. El servicio de libros se dedica solo a la lógica del negocio y a los datos.

# 2. Arquitectura (modelo C4)

El sistema se describe en dos niveles del modelo C4: el contexto, que lo muestra desde fuera, y los contenedores, que muestran lo que hay dentro.

### 2.1 Nivel 1: contexto

Intervienen tres elementos: el bibliotecario, que administra el catálogo; un cliente HTTP (en las pruebas, Postman, Swagger o cURL, aunque podría ser una página web) y el sistema Biblioteca Digital. Quien usa el sistema nunca le habla directamente: lo hace a través de un cliente que envía peticiones HTTP con JSON y un token JWT.

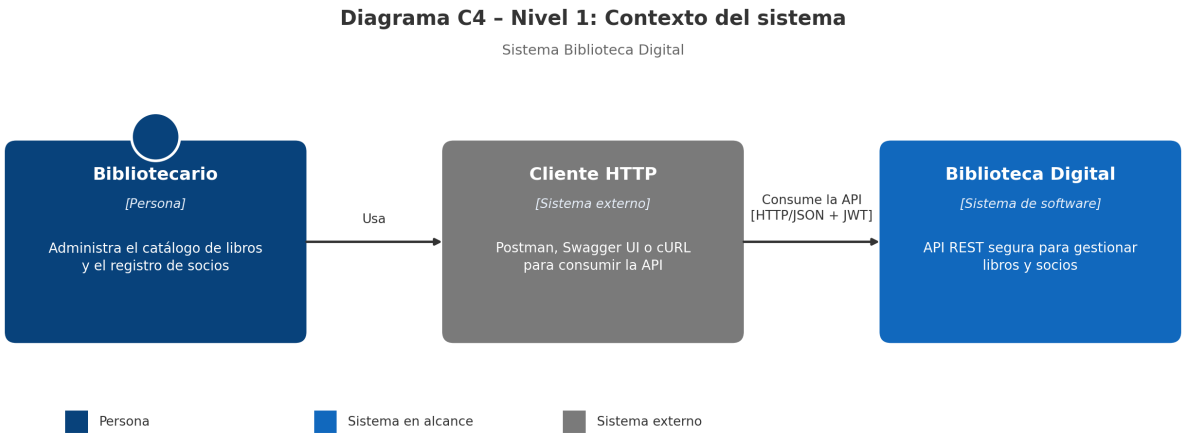



*Figura 1. Diagrama C4, nivel 1 (contexto).*

## 2.2 Nivel 2: contenedores

Dentro del sistema hay tres contenedores Docker unidos por la red bridge *biblioteca-net*. El API Gateway es el único que publica un puerto hacia fuera (8080). Valida el token y reenvía lo que llega a `/api/*` al microservicio, que escucha en el puerto 8000 solo dentro de la red interna. El microservicio guarda los datos en PostgreSQL, que tampoco es accesible desde el equipo anfitrión.

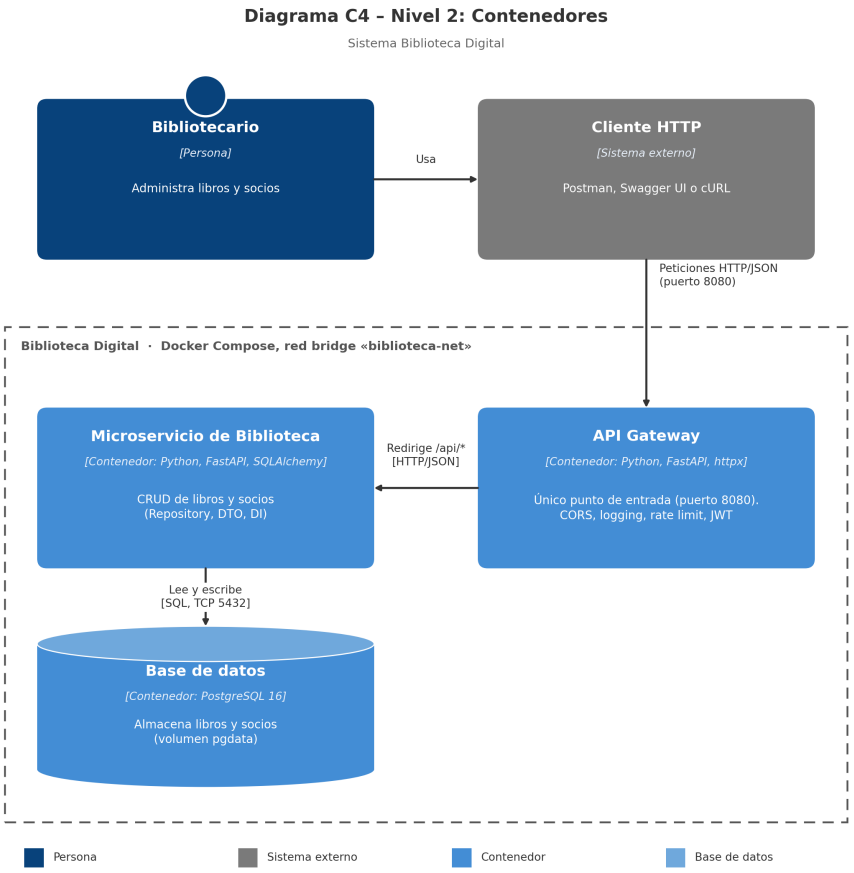



*Figura 2. Diagrama C4, nivel 2 (contenedores).*

## 2.3 Decisiones de diseño

**Gateway en FastAPI.** Se pudo usar Nginx, pero validar JWT y escribir el rate limiting resulta más directo en Python, y los cuatro mecanismos quedan en un solo archivo (`gateway/app/main.py`).

**Microservicio por capas.** Los routers solo manejan HTTP; el acceso a datos está en un repositorio genérico (`repository.py`) y los esquemas de Pydantic hacen de DTO, de modo que los modelos de SQLAlchemy no se exponen. La sesión de base de datos y los repositorios se inyectan con `Depends`.

**Tablas al arrancar.** El servicio crea las tablas con SQLAlchemy al iniciar. Para un proyecto de este tamaño evita montar migraciones, aunque en producción convendría usar Alembic.

## 3. API Gateway: funciones transversales

El gateway implementa cuatro mecanismos, dos más de los que pedía la consigna:

| Mecanismo | Cómo funciona |
|---|---|
| CORS | `CORSMiddleware` permite cualquier origen, método y cabecera (útil en desarrollo; en producción se limitaría a los dominios conocidos) y expone `X-Request-ID`. |
| Logging y trazabilidad | Cada petición lleva un `X-Request-ID`: el que envía el cliente o un UUID generado. Se registran método, ruta, código de respuesta y tiempo en milisegundos, y el mismo id se reenvía al microservicio. |
| Rate limiting | Ventana deslizante en memoria por IP: 60 peticiones por minuto (variable `RATE_LIMIT`). Al superarlo responde 429 con `Retry-After`. `/health` y las peticiones OPTIONS quedan exentas. |
| Validación JWT | `/auth/login` emite un token HS256 con vigencia de 60 minutos. Todas las rutas `/api/*` exigen `Authorization: Bearer`; si falta o es inválido, 401. |

## 4. Documentación de endpoints

Todo se consume a través del gateway, en `http://localhost:8080`. El gateway quita el prefijo `/api` antes de reenviar, así que `/api/books` llega al microservicio como `/books`.

| Método | URL | Descripción | Respuestas |
|---|---|---|---|
| POST | /auth/login | Obtiene un token JWT | 200, 401 |
| GET | /health | Estado del gateway | 200 |
| GET | /api/books | Lista libros (parámetros skip y limit) | 200 |
| GET | /api/books/{id} | Obtiene un libro | 200, 404 |
| POST | /api/books | Crea un libro | 201, 409, 422 |
| PUT | /api/books/{id} | Reemplaza los datos de un libro | 200, 404, 409, 422 |
| DELETE | /api/books/{id} | Elimina un libro | 204, 404 |
| GET | /api/members | Lista socios | 200 |
| GET | /api/members/{id} | Obtiene un socio | 200, 404 |
| POST | /api/members | Crea un socio | 201, 409, 422 |
| PUT | /api/members/{id} | Reemplaza los datos de un socio | 200, 404, 409, 422 |
| DELETE | /api/members/{id} | Elimina un socio | 204, 404 |

Validaciones: título y autor no pueden estar vacíos; el ISBN tiene entre 10 y 20 caracteres; el año va de 0 a 2100; las copias no pueden ser negativas y el correo debe tener formato válido. Si algo falla, el servicio responde 422.

### Ejemplos de peticiones y respuestas

**Login** (`POST /auth/login`)

```
Petición:   {"username": "admin", "password": "admin123"}
Respuesta:  {"access_token": "eyJhbGciOiJIUzI1NiIs...", "token_type": "bearer"}
```

**Crear libro** (`POST /api/books`, respuesta 201)

```
Petición:
{
  "title": "Cien años de soledad",
  "author": "Gabriel García Márquez",
  "isbn": "9780307474728",
  "year": 1967,
  "copies_available": 3
}

Respuesta:
{
  "title": "Cien años de soledad",
  "author": "Gabriel García Márquez",
  "isbn": "9780307474728",
  "year": 1967,
  "copies_available": 3,
  "id": 1,
  "created_at": "2026-10-18T15:02:11.482913Z"
}
```

**Crear socio** (`POST /api/members`, respuesta 201)

```
Petición:   {"full_name": "Brayan Chambi", "email": "brayan@example.com"}
Respuesta:  {"full_name": "Brayan Chambi", "email": "brayan@example.com", "id": 1, "created_at": "..."}
```

**Errores**

```
401 (sin token):      {"detail": "Falta el token Bearer"}
404 (no existe):      {"detail": "Libro no encontrado"}
409 (ISBN repetido):  {"detail": "Ya existe un libro con ese ISBN"}
429 (rate limit):     {"detail": "Demasiadas peticiones, intenta más tarde"}
```

## 5. Contenerización

El gateway y el microservicio tienen su propio Dockerfile, basado en `python:3.12-slim`. Primero se copia `requirements.txt` y se instalan las dependencias, y recién después el código, para que Docker reutilice esa capa mientras solo cambie el código. El proceso corre con un usuario sin privilegios.

El archivo `docker-compose.yml` define tres servicios en la red bridge *biblioteca-net*. El arranque está encadenado con `depends_on` y `condition: service_healthy`: primero PostgreSQL (healthcheck con `pg_isready`), luego el microservicio y al final el gateway. Los dos servicios de Python comprueban su salud llamando a `/health` con `urllib`, porque la imagen slim no trae curl.

Los datos de PostgreSQL viven en el volumen `pgdata` y sobreviven a un `docker-compose down`. Las credenciales y el secreto JWT se leen de variables de entorno (archivo `.env`), con valores por defecto para desarrollo.

## 6. Evidencia de pruebas

Las capturas se tomaron con el sistema levantado con `docker-compose up --build`.

Captura pendiente: docs/capturas/01_compose_ps.png

*Captura 1. Los tres contenedores en ejecución y en estado healthy (docker-compose ps).*

Captura pendiente: docs/capturas/02_login.png

*Captura 2. POST /auth/login: respuesta 200 con el token JWT.*

Captura pendiente: docs/capturas/03_sin_token.png

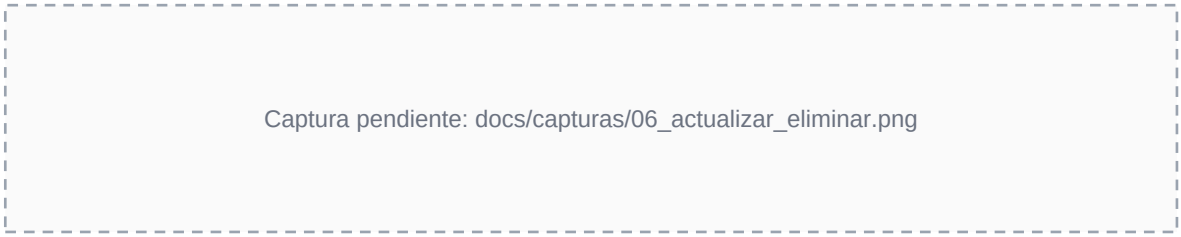
*Captura 3. GET /api/books sin token: el gateway responde 401.*

Captura pendiente: docs/capturas/04_crear_libro.png

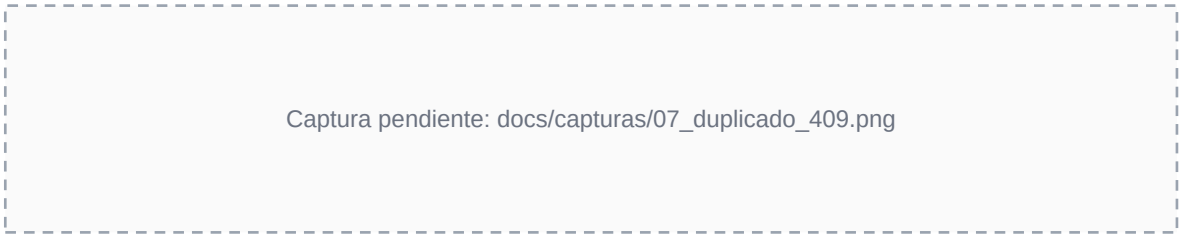
*Captura 4. POST /api/books: libro creado, respuesta 201.*

Captura pendiente: docs/capturas/05_listar_obtener.png

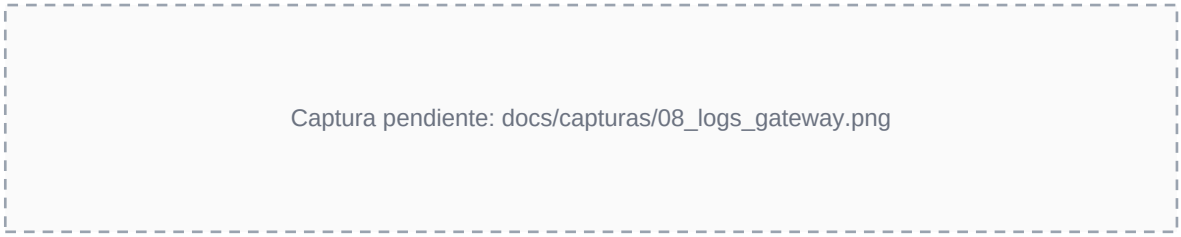
*Captura 5. GET /api/books y GET /api/books/{id}: respuestas 200.*




*Captura 6. PUT /api/books/{id} (200) y DELETE /api/books/{id} (204).*




*Captura 7. Intento de registrar un ISBN repetido: respuesta 409.*




*Captura 8. Logs del gateway (docker-compose logs gateway) con el rid de cada petición.*

## 7. Conclusiones y limitaciones

La arquitectura cumple lo planteado: un único punto de entrada, un servicio con CRUD completo sobre dos entidades, persistencia en PostgreSQL y todo el conjunto levantándose con un solo comando.

Quedan algunos límites conocidos. El rate limiting guarda los contadores en memoria, así que no se compartiría entre varias réplicas del gateway (para eso haría falta algo como Redis). Hay un solo usuario administrador definido por variables de entorno, CORS está abierto a cualquier origen y todavía no existe el registro de préstamos. Son mejoras razonables para una siguiente versión.